

CMP5104 RECOMMENDER SYSTEMS

Assignment 2 - Report

Esranur Aygün – 1904469

Date:14/12/2025

Introduction and Scope

The goal of this assignment was to build and optimize a recommender system to minimize prediction error. I used PyTorch on Google Colab using the *MovieLens 100K dataset* on this assignment.

Environmental Setup and Data Processing

To maximize training speed, right after loading the data, I did all the pre-processing. I converted all the user and movie IDs into simple, consecutive numbers and then turned the entire training and test sets into PyTorch Tensors stored directly on the GPU. This "zero-latency" pipeline meant the model didn't have to wait for data, letting me train over 100 different configurations quickly and without crashing.

I defined five distinct model classes to compare different recommendation strategies: bias only model, matrix factorization model, matrix factorization with bias model, neural collaborative filtering model, NCF without bias model.

Main Grid Search

To find the optimal parameters, I ran a huge Grid Search across 125 different combinations with given parameters: **embedding_dims** = [8, 16, 32, 64, 128], **learning_rates** = [0.001, 0.005, 0.01, 0.05, 0.1], **regularizations** = [0.0, 1e-6, 1e-5, 1e-4, 1e-3]. I used three advanced optimization techniques rather than standard training loops:

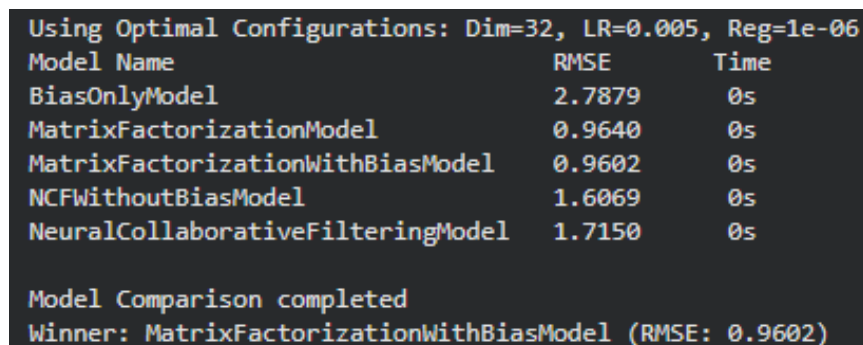
- **AdamW optimizer:** AdamW decouples weight decay (regularization) from the gradient update step, which prevents the model from confusing "regularization penalties" with actual "learning directions," leading to better generalization on sparse data.
- **ReduceLROnPlateau Scheduler:** This acted as a dynamic "brake system": whenever the validation error stopped dropping for 2 epochs, the scheduler automatically cut the learning rate in half. This allowed the model to make large, fast jumps early on, but then slow down to fine-tune the weights once it got close to the optimal solution.
- **Early Stopping** with a patience limit of 3. If a model failed to improve for three consecutive epochs, training was terminated immediately to save computation time by preventing wasting GPU cycles on "dead" configurations.

The search was a success. It pinpointed a winning configuration: an embedding dimension of 32, a learning rate of 0.005, and very light regularization. This setup achieved a low RMSE of 0.9533. The result told me that a model of moderate complexity, tuned with a specific learning speed, performed the best.

Model Comparison

For the final model benchmarking, all five architectures were trained for a maximum of 100 epochs using the optimal hyperparameters (Dim=32, Reg=1e-06) identified by the Grid Search.

To ensure a fair comparison, I used the same AdamW optimizer, scheduler, and early stopping rules (patience=5) for every model. I made one necessary adjustment for the neural network models (NCF): I lowered their learning rate to 0.001, compared to the 0.005 used for the Matrix Factorization models. This change was required because the neural networks were unstable and failed to learn at the higher speed.

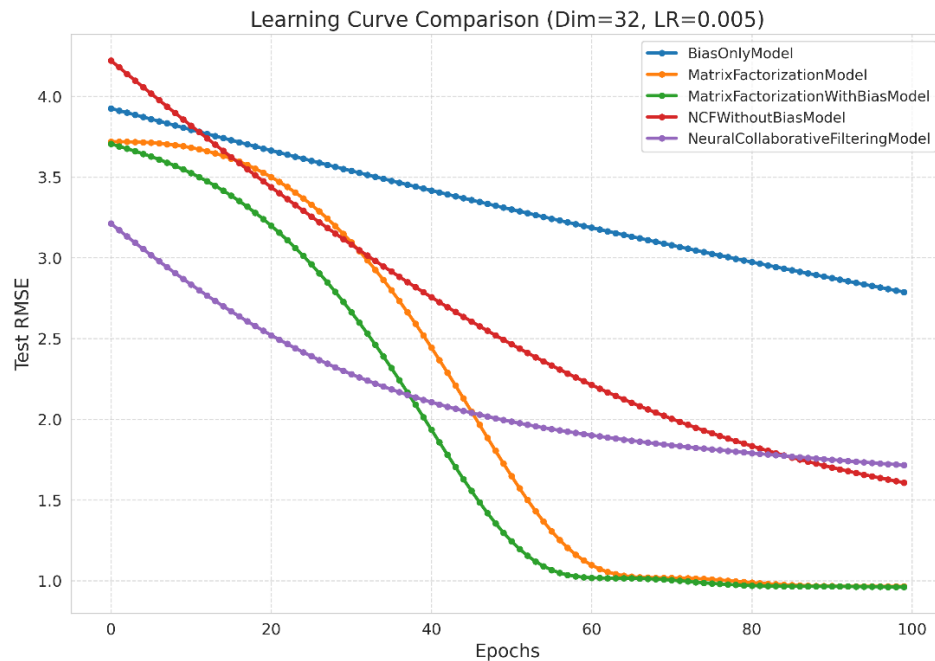
A terminal window with a dark background and light-colored text. It displays the results of a model comparison. At the top, it states the optimal configurations used: Dim=32, LR=0.005, and Reg=1e-06. Below this is a table with three columns: Model Name, RMSE, and Time. Five models are listed: BiasOnlyModel, MatrixFactorizationModel, MatrixFactorizationWithBiasModel, NCFWithoutBiasModel, and NeuralCollaborativeFilteringModel. The MatrixFactorizationWithBiasModel has the lowest RMSE of 0.9602. At the bottom, it says 'Model Comparison completed' and 'Winner: MatrixFactorizationWithBiasModel (RMSE: 0.9602)'.

Using Optimal Configurations: Dim=32, LR=0.005, Reg=1e-06		
Model Name	RMSE	Time
BiasOnlyModel	2.7879	0s
MatrixFactorizationModel	0.9640	0s
MatrixFactorizationWithBiasModel	0.9602	0s
NCFWithoutBiasModel	1.6069	0s
NeuralCollaborativeFilteringModel	1.7150	0s
Model Comparison completed		
Winner: MatrixFactorizationWithBiasModel (RMSE: 0.9602)		

Learning Curve Analysis

I plotted the validation error for all five models across all 100 epochs to visualize their learning journeys.

- The MatrixFactorizationWithBiasModel (Green line) demonstrated the fastest convergence, dropping steeply in the first 20 epochs and stabilizing at the lowest error.
- The NCF models (Red and Purple lines) plateaued early at a much higher error rate (~1.6), confirming that the neural network architecture struggled to find meaningful patterns in this sparse data compared to the dot-product approach.
- The BiasOnlyModel (Blue line) showed almost no improvement over time, proving it wasn't capable of real personalization.



5-Fold Cross-Validation

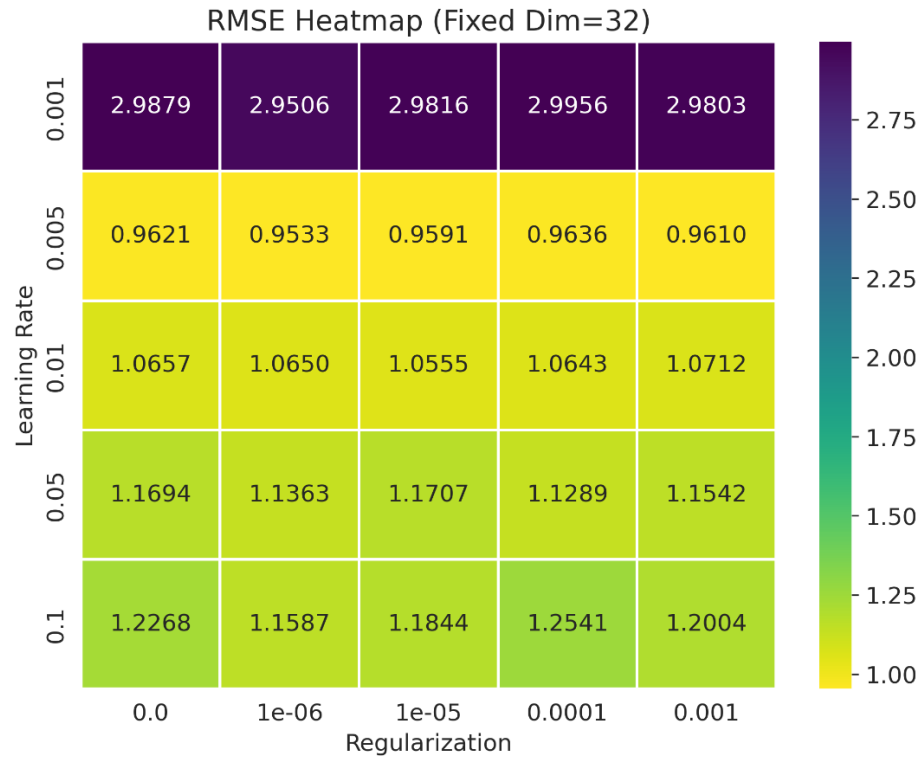
To verify the best parameters, I performed 5-Fold Cross-Validation. I retrained the model (Dim=32, LR=0.005) from scratch five separate times, each on a different pre-defined split of the data (u1 through u5). The model achieved a final average RMSE of 0.9521, which is very close to Grid Search result (0.9533).

```
Model: MatrixFactorizationWithBiasModel
Config: Dim=32, LR=0.005, Reg=1e-06
-----
Fold  RMSE    Time
Fold 1 0.9502    1s
Fold 2 0.9362    1s
Fold 3 1.0016    0s
Fold 4 0.9373    1s
Fold 5 0.9353    1s
cross-validation completed
Final Report Score is 0.9521 ± 0.0253
```

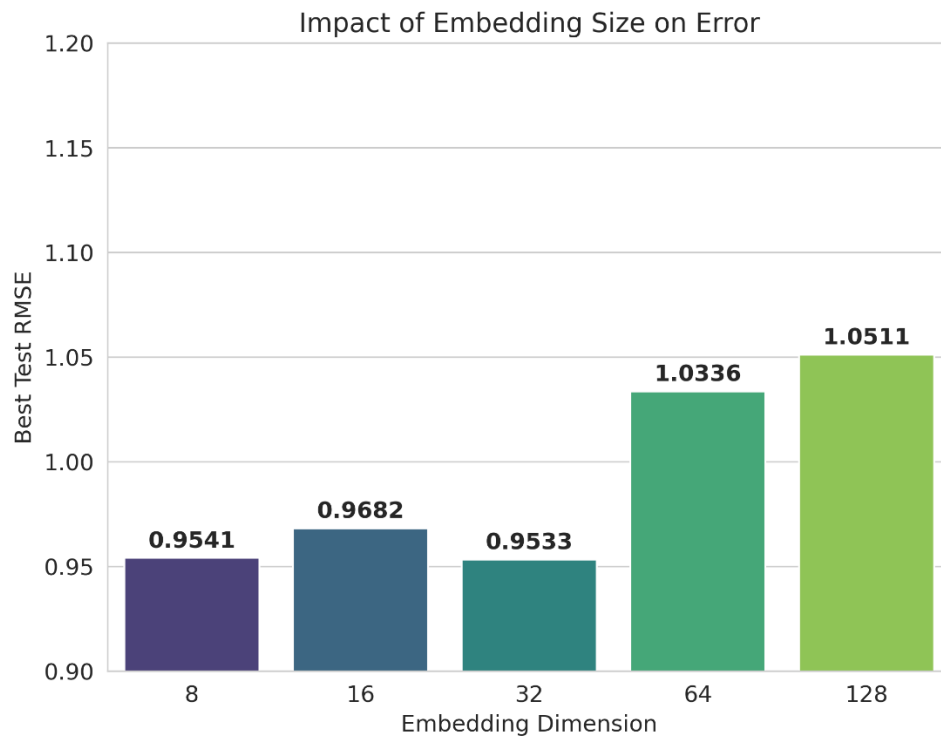
FINAL VISUALIZATION

Three key visualizations were produced to summarize the project findings conclusively.

- **Hyperparameter Heatmap:** This showed the Learning Rate was the most important setting. A rate of 0.005 (yellow/green zone) was the "stable region" where learning worked. Rates that were too low (purple zones) caused failure.



- **Embedding Size Analysis:** The bar chart showed a clear "U-shape". Performance was best with 32 factors. Making the model bigger (64 or 128 factors) made the error worse, which is a sign of overfitting.



- **Model Comparison:** This chart made the winner obvious. It shows a huge gap between our best model (error ~0.96) and the neural network baselines (error 1.60+). It definitively answers that for this dataset, Matrix Factorization with Bias is the most effective architecture.

